

The image features a low-angle, upward-looking perspective of a dense urban skyline. Several tall skyscrapers are visible, with one prominent building on the left having a textured, metallic facade. In the foreground, a modern, curved glass and metal structure, possibly a transit station or public building entrance, is partially visible. A tall, thin antenna or tower rises from one of the buildings in the background. The sky is a clear, pale blue. Overlaid in the center is the SENAI logo, which consists of the word "SENAI" in a bold, white, sans-serif font, set against a solid red rectangular background. The logo is slightly offset to the right.

**SENAI**



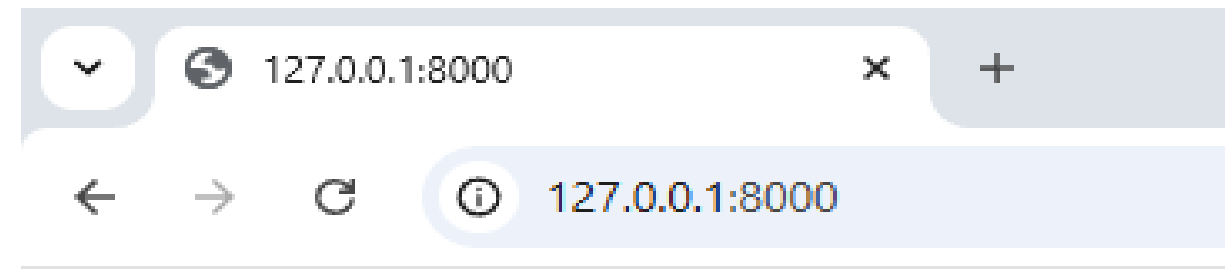
# **App's no django**

# Django – App's

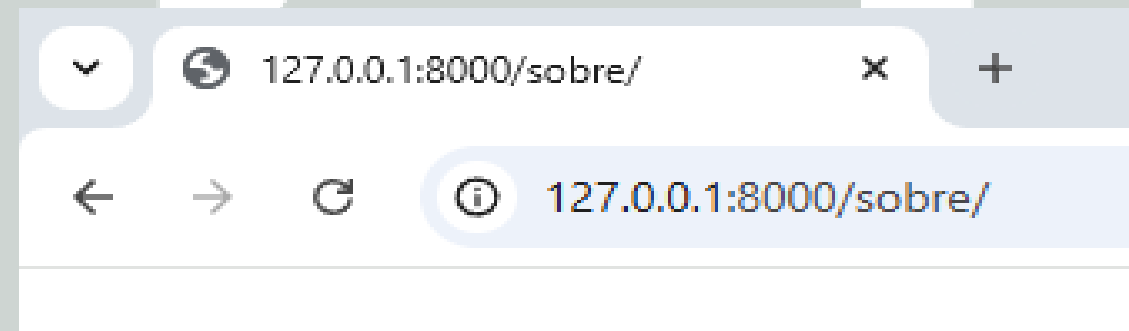
Suponha que você

está desenvolvendo no  
Django três página na  
sua aplicação

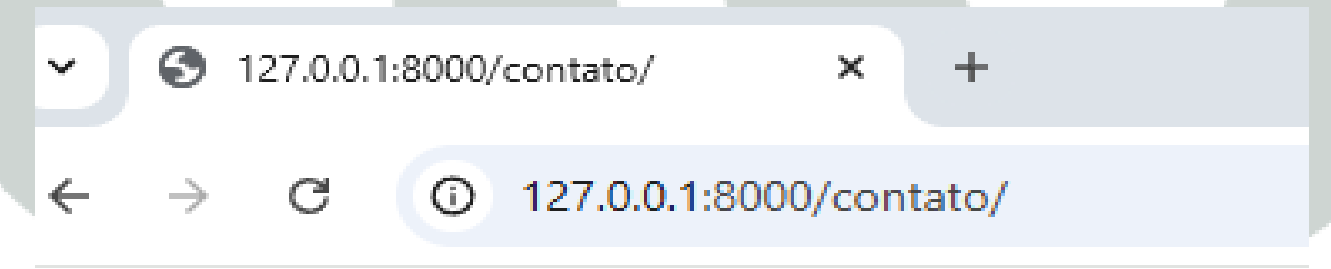
- *Home*
- *Sobre*
- *Contatos*



## home



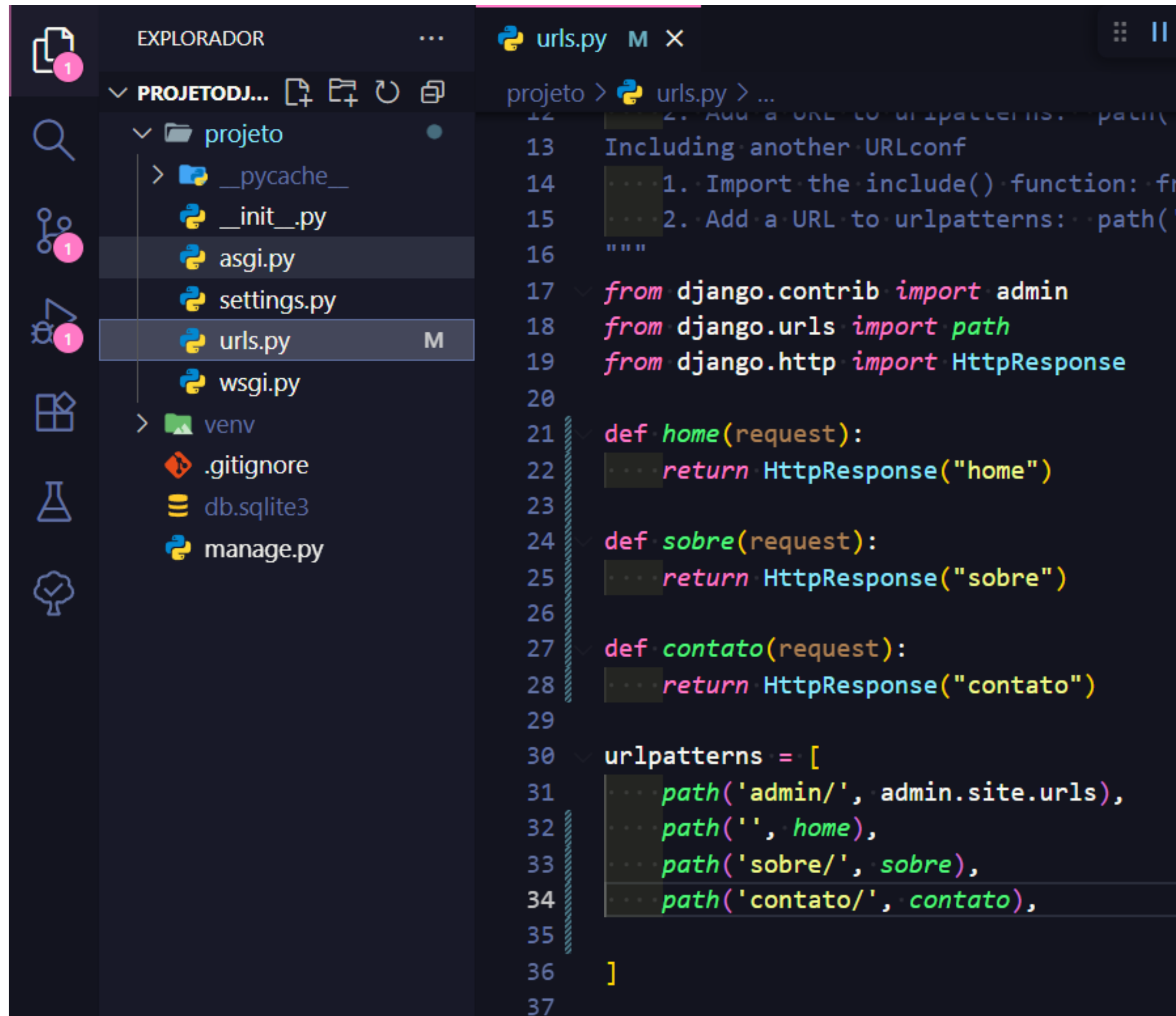
## sobre



## contato

# Django – App's

No código, fica assim:



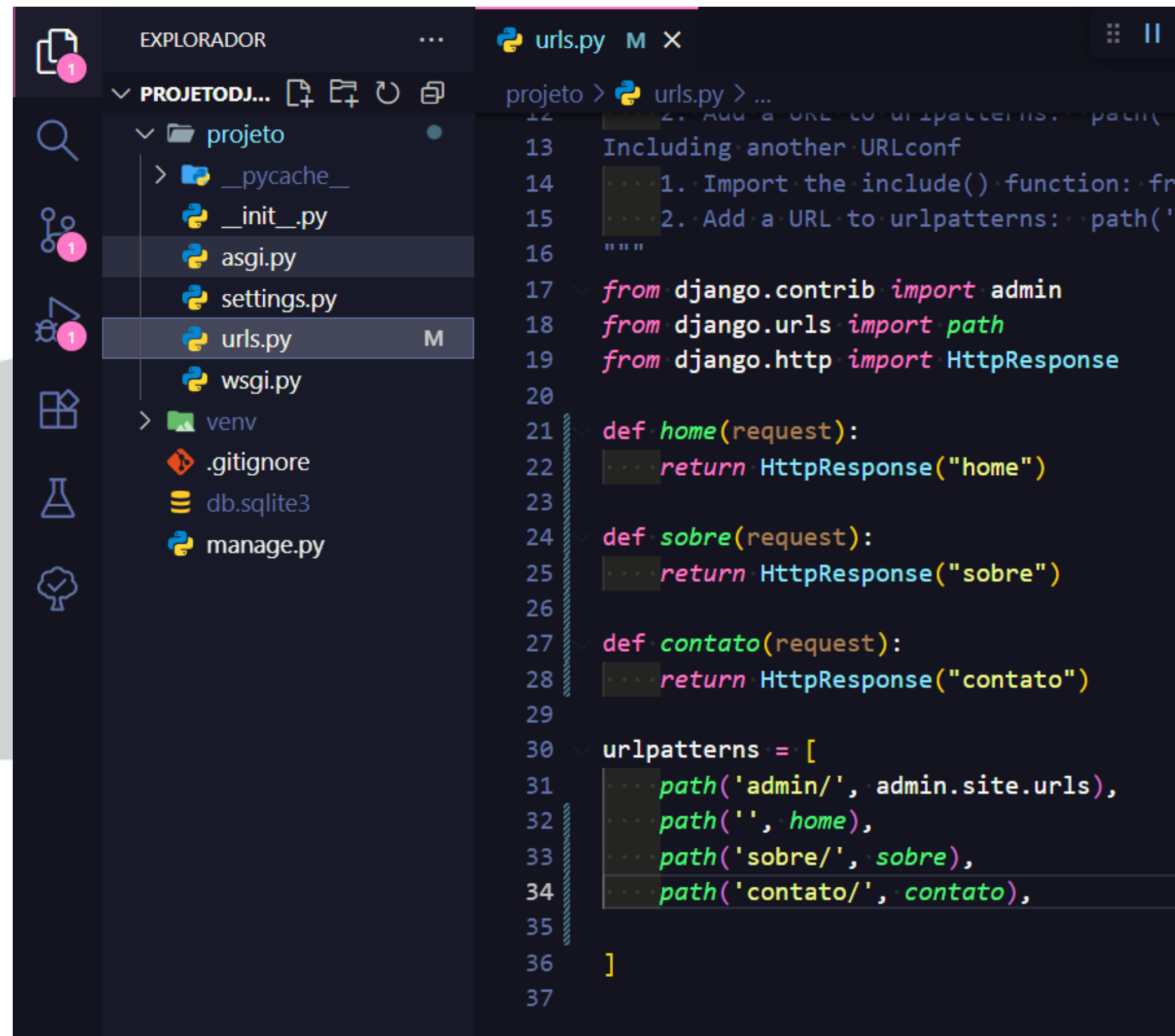
The image shows a code editor interface with a dark theme. On the left, the 'EXPLORADOR' (Explorer) pane displays the project structure for 'PROJETODJ...'. The 'projeto' folder is expanded, showing files: '\_\_pycache\_\_', '\_\_init\_\_.py', 'asgi.py', 'settings.py', 'urls.py' (marked with an 'M'), and 'wsgi.py'. Below this are 'venv', '.gitignore', 'db.sqlite3', and 'manage.py'. On the right, the 'urls.py' file is open, showing Python code for URL patterns. The code includes imports for 'admin', 'path', and 'HttpResponse' from Django. It defines three view functions: 'home', 'sobre', and 'contato', each returning an 'HttpResponse'. The 'urlpatterns' list is defined at the bottom, including paths for 'admin/', the root path (calling 'home'), 'sobre/', and 'contato/' (calling 'contato').

```
12 2. Add a URL to urlpatterns: path(
13 Including another URLconf
14 ... 1. Import the include() function: fr
15 ... 2. Add a URL to urlpatterns: path('
16 """
17 from django.contrib import admin
18 from django.urls import path
19 from django.http import HttpResponse
20
21 def home(request):
22     return HttpResponse("home")
23
24 def sobre(request):
25     return HttpResponse("sobre")
26
27 def contato(request):
28     return HttpResponse("contato")
29
30 urlpatterns = [
31     path('admin/', admin.site.urls),
32     path('', home),
33     path('sobre/', sobre),
34     path('contato/', contato),
35
36 ]
37
```

# Django – App's

Isso é inviável, pois uma página existe muitos recursos e codar dentro do arquivo `urls.py` fica complicado.

Para isso vamos utilizar o app's do Django.



The screenshot shows a code editor with two panels. The left panel, titled 'EXPLORADOR', displays the file structure of a project named 'PROJETODJ...'. It shows a folder 'projeto' containing files like `__pycache__`, `__init__.py`, `asgi.py`, `settings.py`, `urls.py` (highlighted with a blue bar and a pink '1' icon), and `wsgi.py`. Below the 'projeto' folder are other files: `venv`, `.gitignore`, `db.sqlite3`, and `manage.py`. The right panel shows the content of `urls.py`. It includes imports for `admin`, `path`, and `HttpResponse` from Django. There are three view functions: `home`, `sobre`, and `contato`, each returning an `HttpResponse`. The `urlpatterns` list is defined at the bottom, including paths for the admin site and the three views.

```
12 2. Add a URL to urlpatterns: path(
13 Including another URLconf
14 ... 1. Import the include() function: fr
15 ... 2. Add a URL to urlpatterns: path('
16 """
17 from django.contrib import admin
18 from django.urls import path
19 from django.http import HttpResponse
20
21 def home(request):
22     return HttpResponse("home")
23
24 def sobre(request):
25     return HttpResponse("sobre")
26
27 def contato(request):
28     return HttpResponse("contato")
29
30 urlpatterns = [
31     path('admin/', admin.site.urls),
32     path('', home),
33     path('sobre/', sobre),
34     path('contato/', contato),
35
36 ]
37
```



# **App's no Django App receitas**

# Django – App's

Em Django, "apps" (aplicativos) são partes separadas de um projeto que lidam com uma funcionalidade específica, como um blog, autenticação ou pagamentos. Essa estrutura modular facilita a organização, reutilização de código e o desenvolvimento de aplicações maiores. Um projeto pode conter vários apps, mas cada app é responsável por uma única função.

django

# Django – App's

Como funcionam:

Projeto vs. App: Um projeto Django é a configuração completa, enquanto um app é um componente funcional dentro desse projeto.

- Modularidade: A divisão em apps permite que funcionalidades sejam desenvolvidas e mantidas de forma isolada. Por exemplo, um aplicativo de e-commerce pode ter apps para clientes, produtos e pedidos.
- Reutilização: Um app pode ser reutilizado em outros projetos, facilitando a criação de aplicações semelhantes.



# Django – App's

Como criar um app:

- Comando: Abra o terminal, navegue até a raiz do seu projeto Django e execute: `python manage.py startapp <nome_do_app>`.
- Exemplo: Para criar um app chamado blog, o comando seria `python manage.py startapp blog`

django

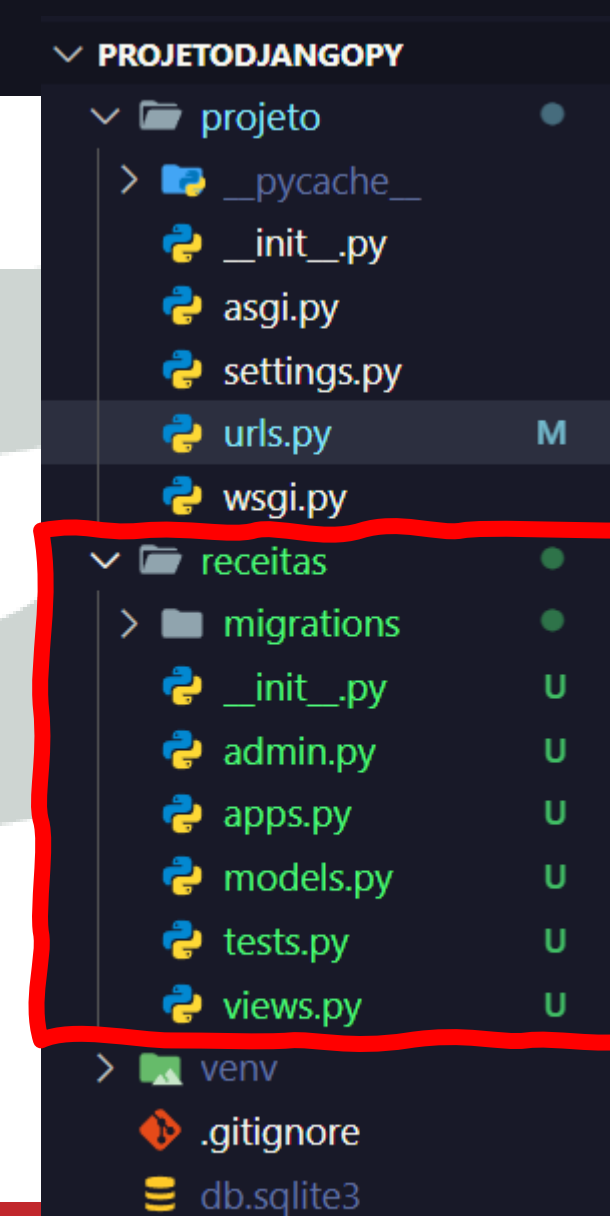
# Django – App's

Vamos criar um app de receitas.

No terminal rodar o código *python manage.py startapp receitas*

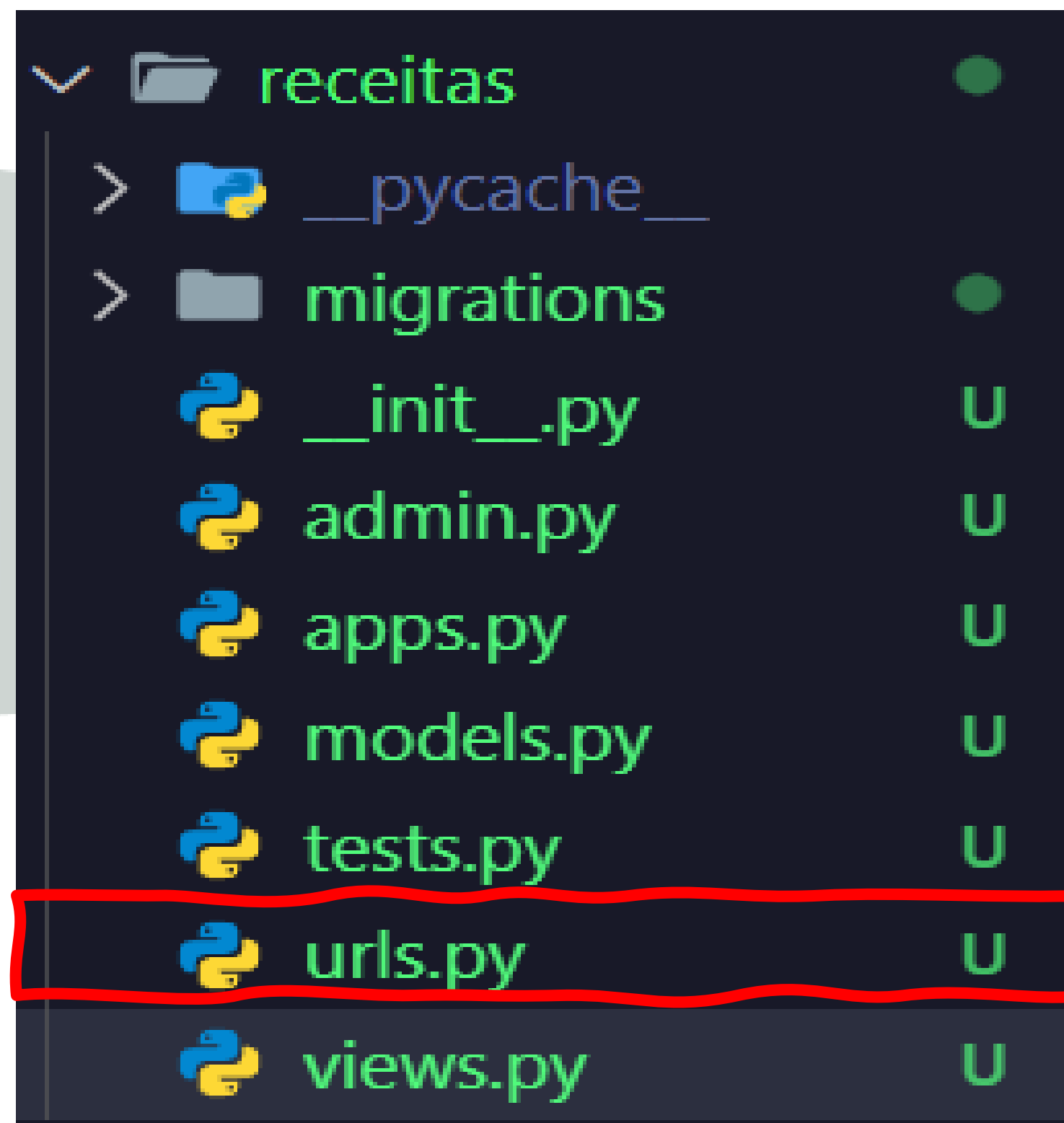
```
25 - 2\FIC\Python como framework\ProjetoDjangoPY\manage.py' 'runserver' ...  
• (venv) PS F:\Senai\2025 - 2\FIC\Python como framework\ProjetoDjangoPY> python manage.py startapp receitas  
✧ (venv) PS F:\Senai\2025 - 2\FIC\Python como framework\ProjetoDjangoPY> 
```

Será criado uma nova pasta dentro do projeto do nosso  
app de receitas



# Django – App's

Vamos migrar nossas url's para dentro do nosso app. Para isso criar um arquivo *urls.py* dentro da pasta *receitas*





# Django – App's

Dentro do arquivo urls.py, inserir o seguinte código

```
receitas >  urls.py > ...  
1  from django.urls import path  
2  from receitas.views import home, sobre, contato  
3  
4  urlpatterns = [  
5      path('', home),  
6      path('sobre/', sobre),  
7      path('contato/', contato),  
8  ]
```

# Django – App's

Dentro do arquivo views.py, inserir o seguinte código

```
receitas >  views.py >  contato  
1  from django.shortcuts import render  
2  from django.http import HttpResponseRedirect  
3  
4  # Create your views here.  
5  def home(request):  
6      ... return HttpResponseRedirect("home url")  
7  
8  def sobre(request):  
9      ... return HttpResponseRedirect("sobre url")  
10  
11  def contato(request):  
12      ... return HttpResponseRedirect("contato url")
```

Trocar as strings das  
respostas http para testar o  
código

# Django – App's

Dentro do arquivo urls.py da pasta projeto, inserir o seguinte código

```
15     ... 2. Add a URL to urlpatterns: path('blog',
16     """
17     from django.contrib import admin
18     from django.urls import path, include
19
20     urlpatterns = [
21         path('admin/', admin.site.urls),
22         path('', include('receitas.urls')),
23     ]
24
```

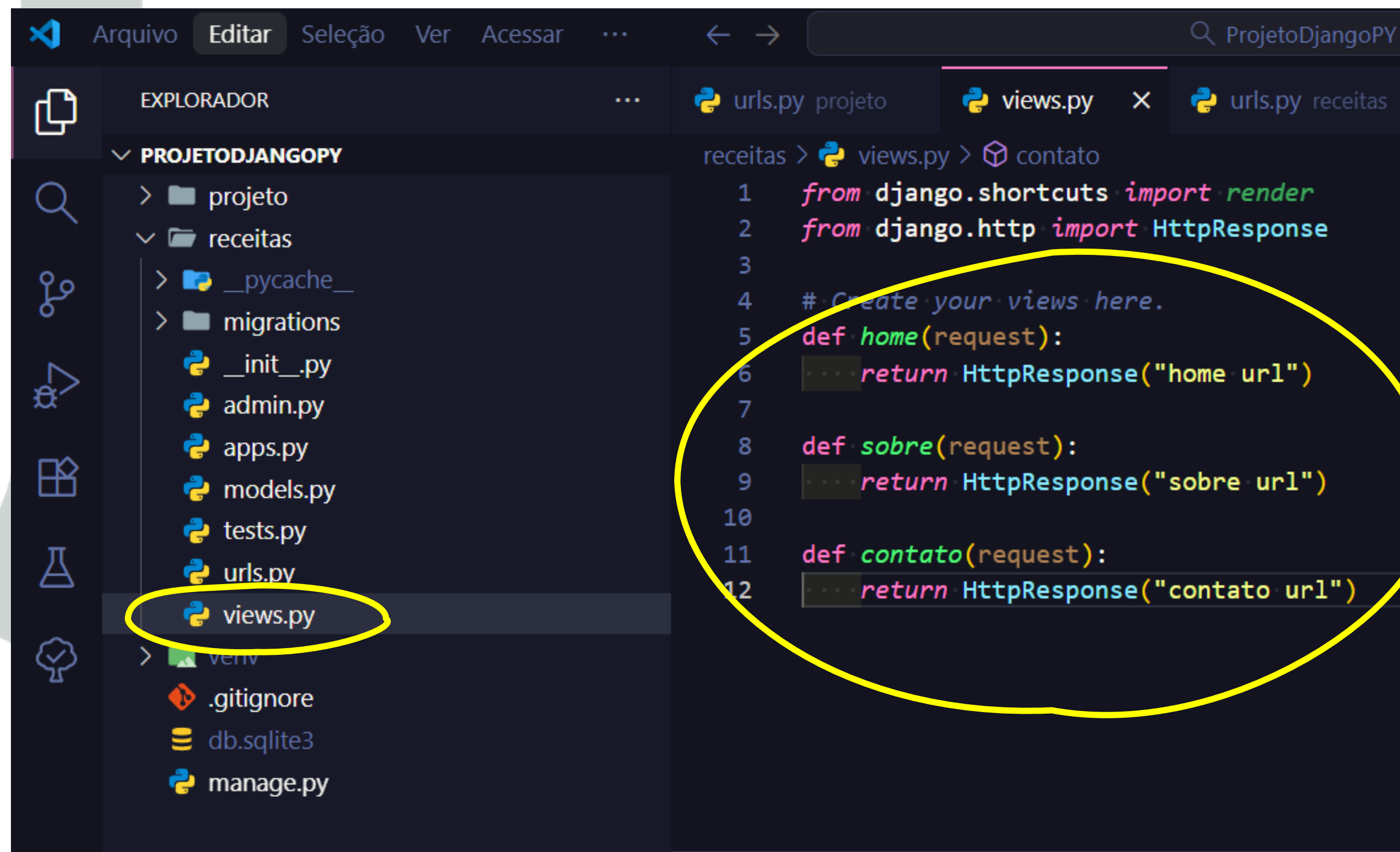
Essa função permite  
inserir vários urls dentro  
do mesmo caminho  
(path)



# **HTML no Django**

# Django – HTML

No views.py as funções colocadas ficam inviáveis

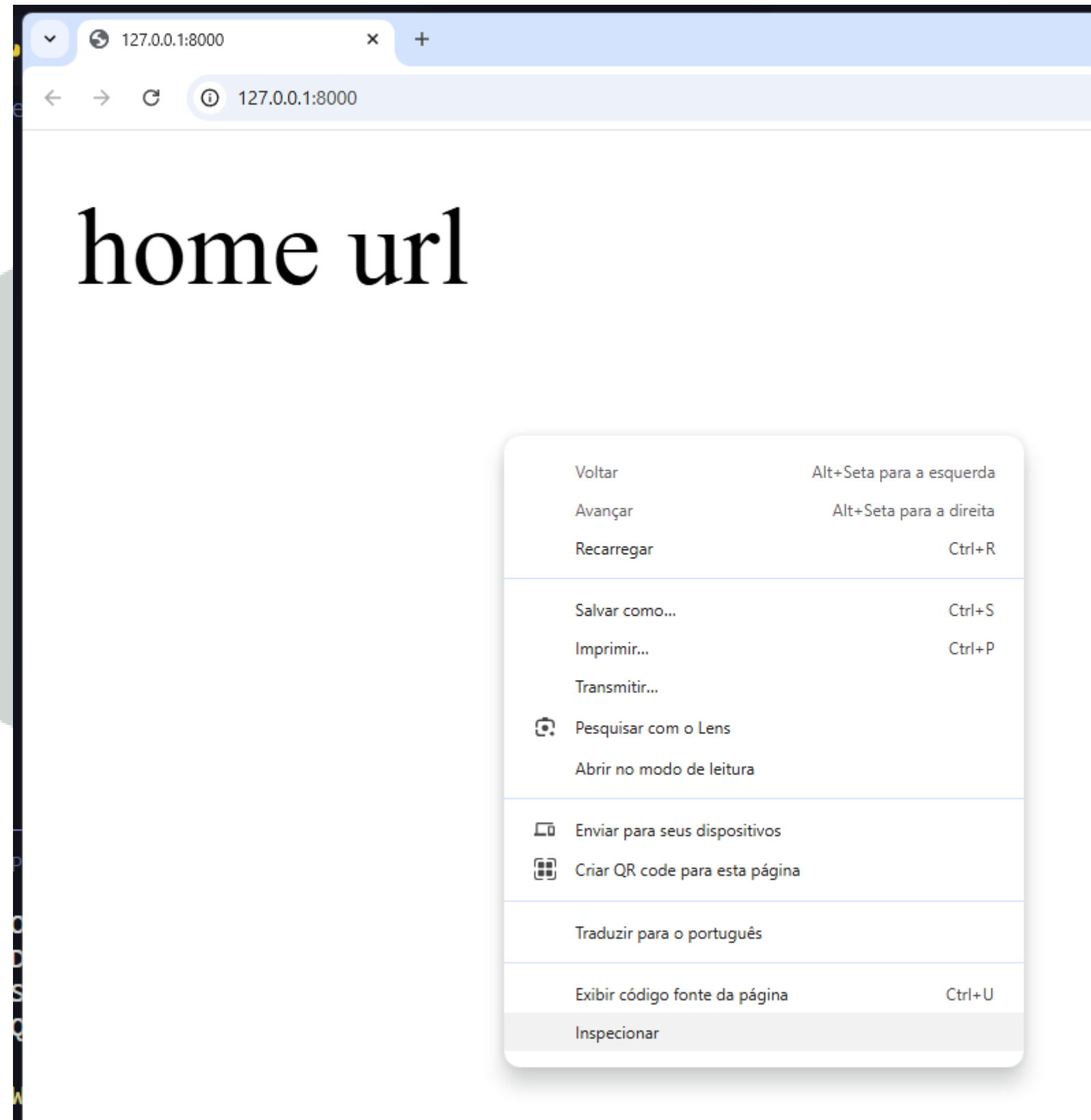


The screenshot displays a code editor interface for a Django project named 'ProjetoDjangoPY'. The left sidebar shows the 'EXPLORADOR' (Explorer) view with the project structure. The 'views.py' file is highlighted in the file explorer. The main editor window shows the content of 'views.py' with the following code:

```
1 from django.shortcuts import render
2 from django.http import HttpResponse
3
4 # Create your views here.
5 def home(request):
6     return HttpResponse("home url")
7
8 def sobre(request):
9     return HttpResponse("sobre url")
10
11 def contato(request):
12     return HttpResponse("contato url")
```

# Django – HTML

Percebe que ao verificar o código fonte da página, nota-se que está escrito nenhum código em html

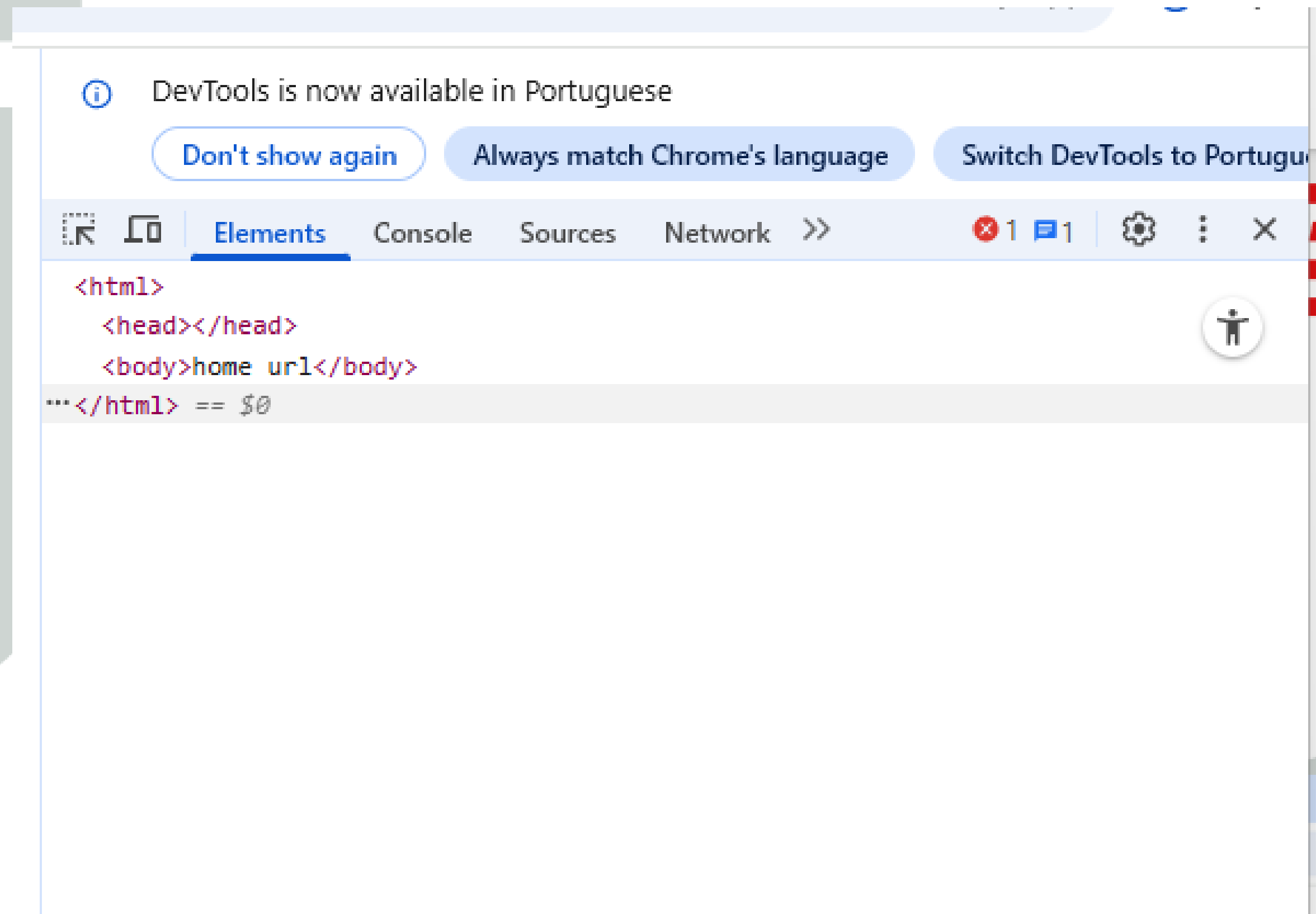


Quebra de linha ☐

1 home url

# Django – HTML

Mas ao inspecionar a página verifica que tem um código html, isso ocorre por que o navegador corrige isso





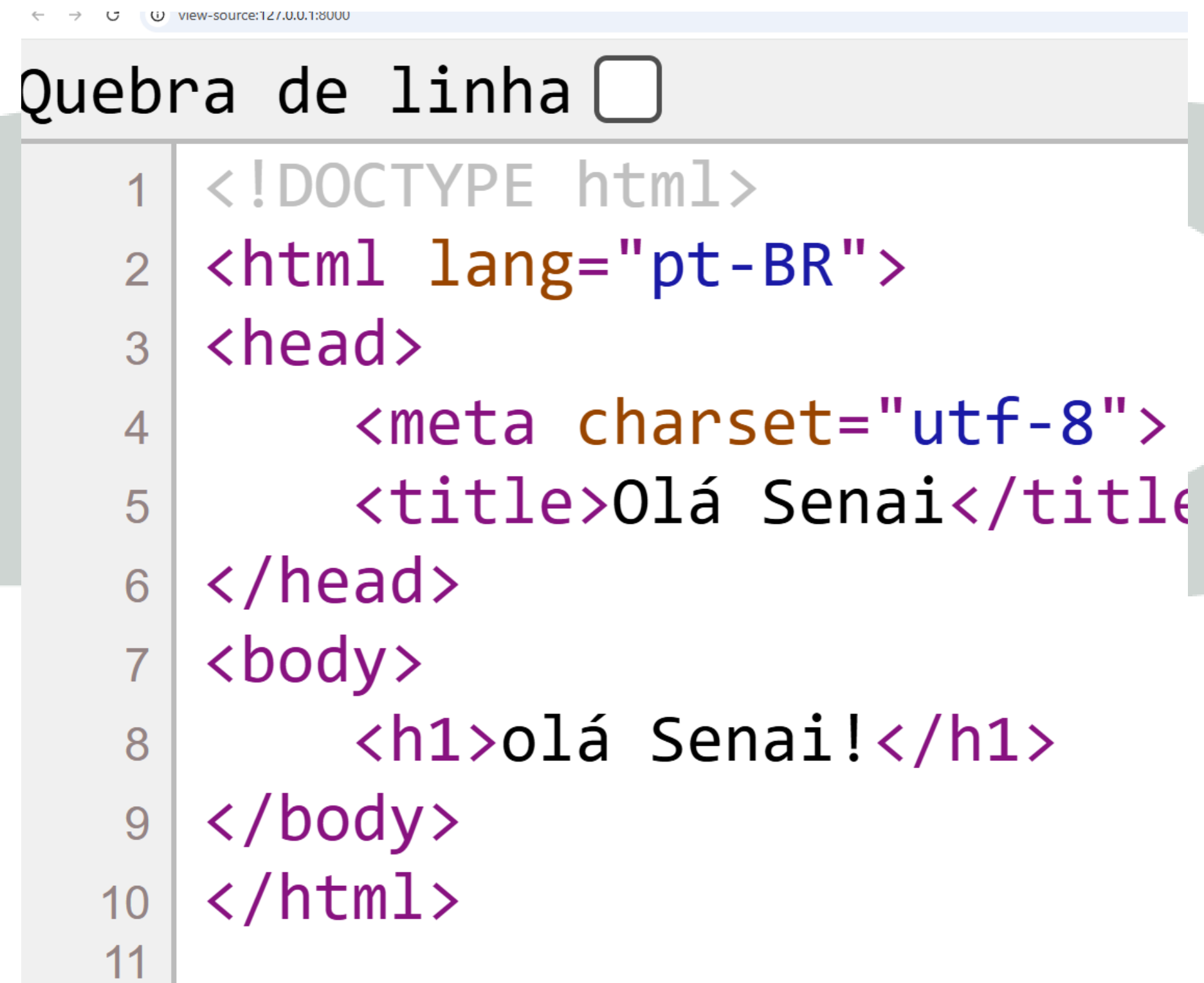
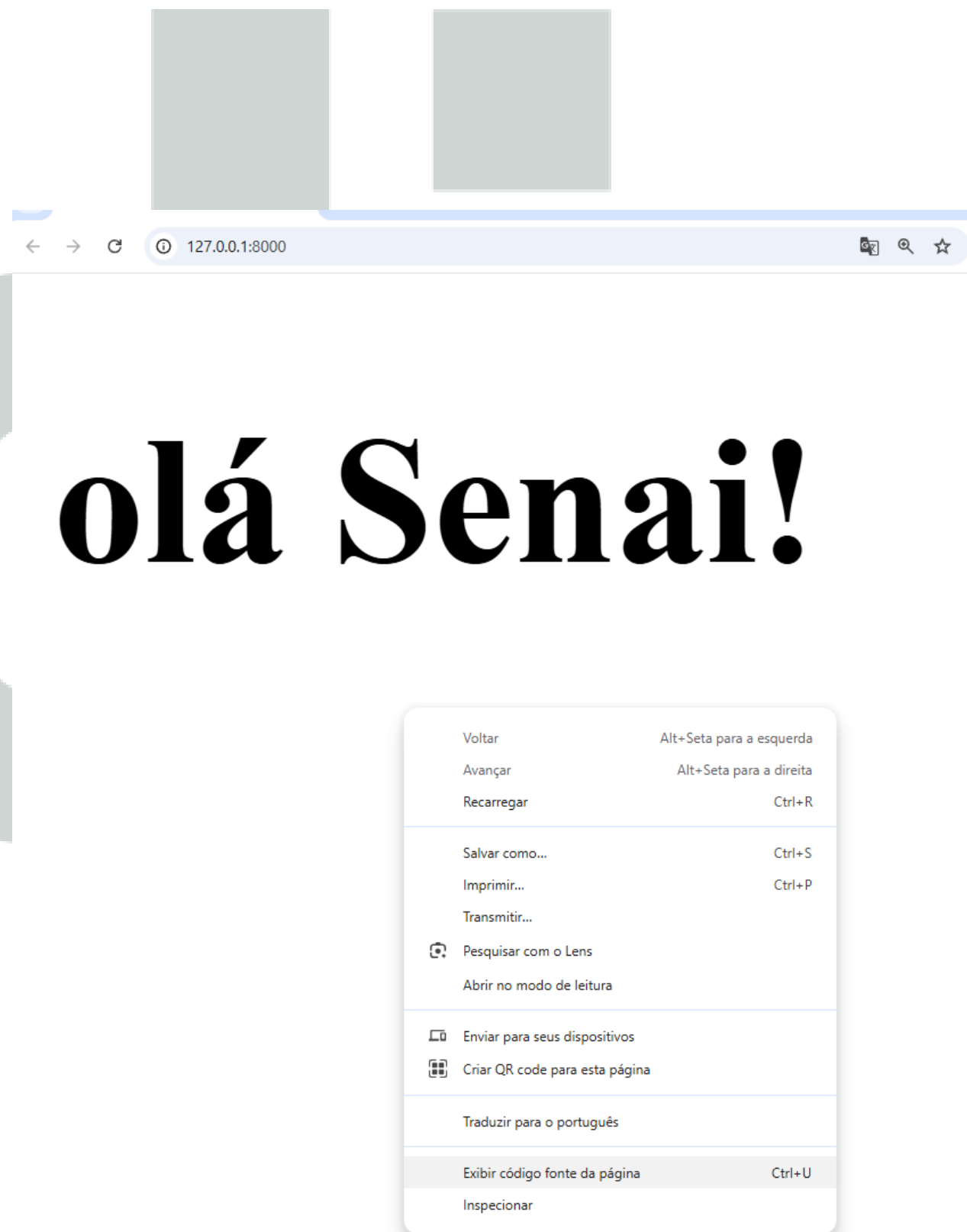
# Django – HTML

Para inserir código HTML na página, teria inserir o código dentro da função, desta forma:

```
3
4 # Create your views here.
5 def home(request):
6     return HttpResponse(''<!DOCTYPE html>
7     <html lang="pt-BR">
8     <head>
9         <meta charset="utf-8">
10        <title>Olá Senai</title>
11    </head>
12    <body>
13        <h1>olá Senai!</h1>
14    </body>
15    </html>
16    ''')
17
```

# Django – HTML

Agora ao verificar o código fonte da página temos um código HTML. Porém sabemos que o código HTML de uma página é muito extenso, tornando o ato inviável.



# Django – HTML

Para melhorar isso vamos utilizar a nossa função render, para renderizar o código HTML na página.

```
receitas >  views.py >  home
1  from django.shortcuts import render
2  from django.http import HttpResponse
3
4  # Create your views here.
5  def home(request):
6      ... return HttpResponse(''<!DOCTYPE html>
7      <html lang="pt-BR">
8      <head>
9      ... <meta charset="utf-8">
10     ... <title>Olá Senai</title>
11     </head>
12     <body>
13     ... <h1>olá Senai!</h1>
14     </body>
15     </html>
16     ''')
17
```

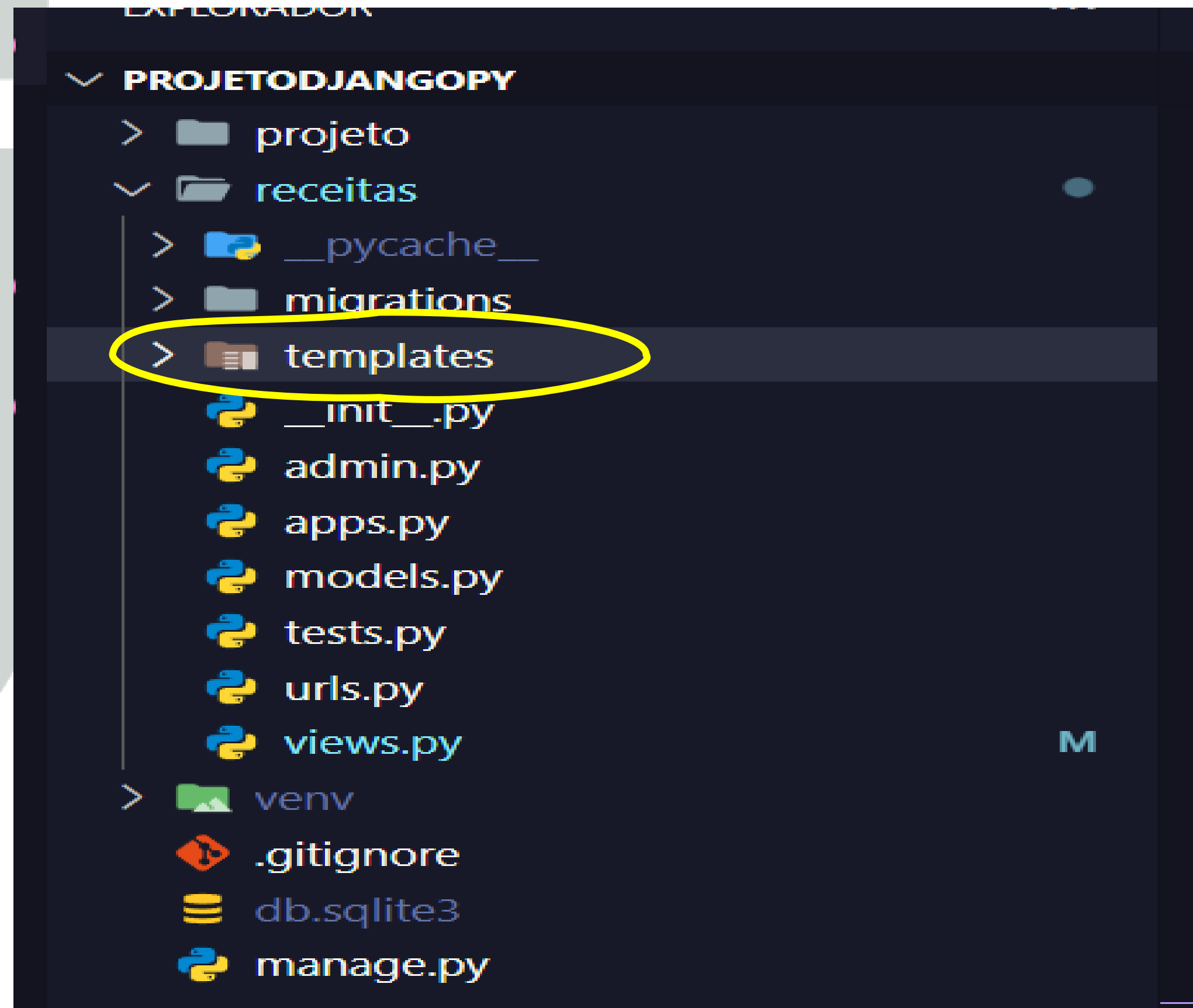
Vamos mudar a função e criar um código html.

```
from django.shortcuts import render
from django.http import HttpResponse

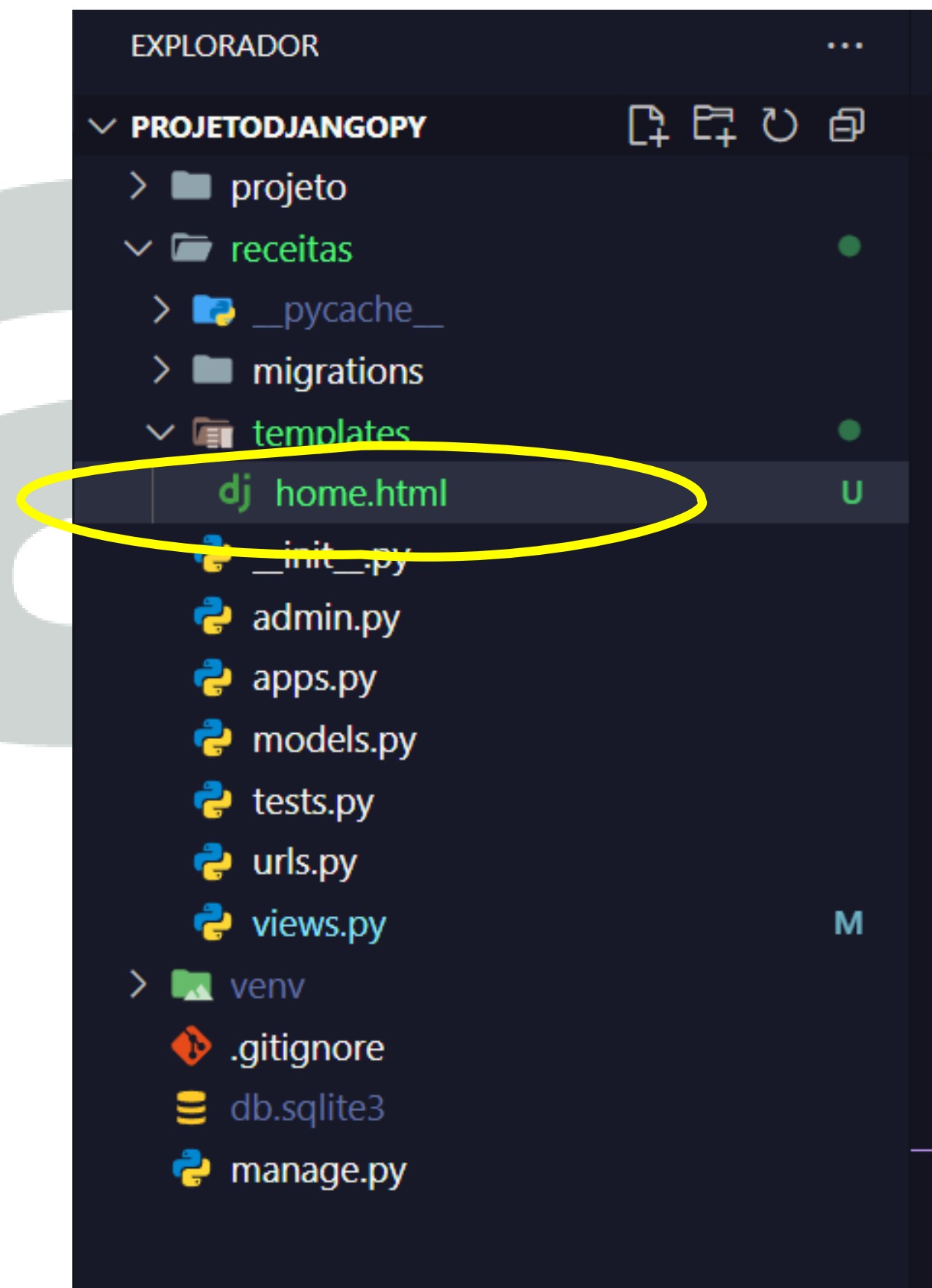
# Create your views here.
def home(request):
    ... return render(request, 'home.html')
```



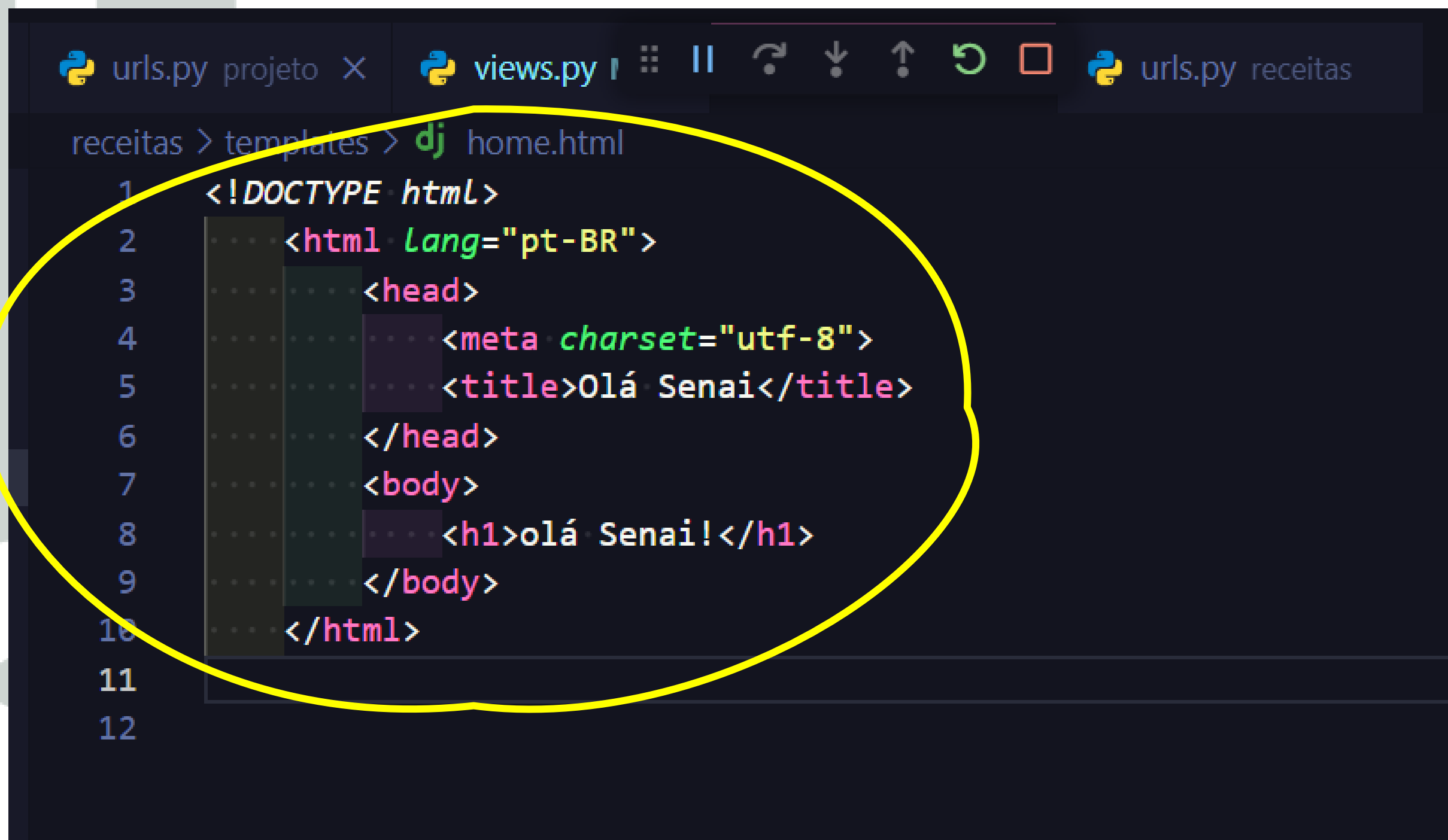
Vamos criar uma pasta com o nome *templates*



Dentro da pasta, vamos criar um arquivo chama *home.html*

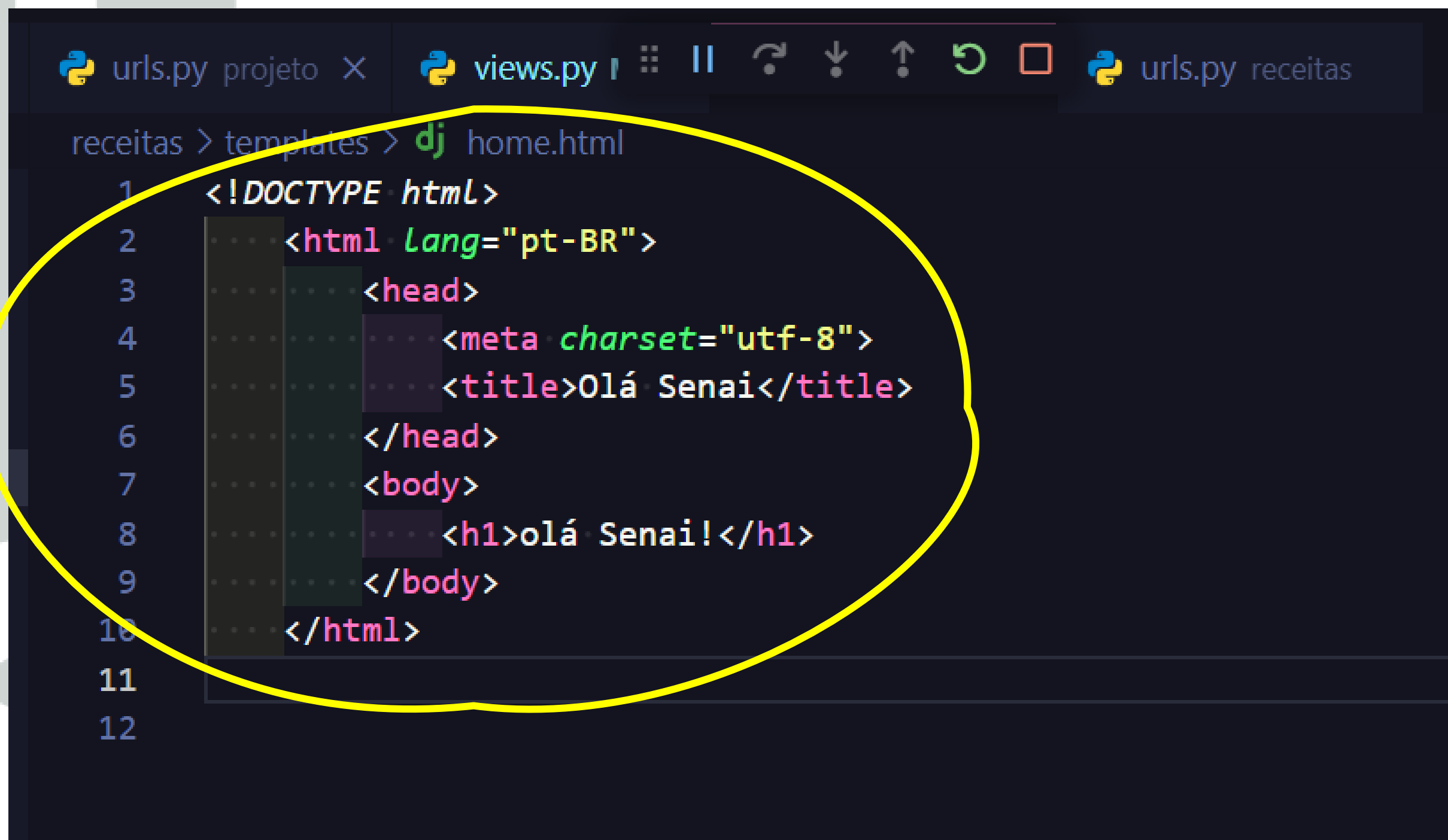


Dentro deste arquivo vamos codar o código html



```
receitas > templates > dj home.html
1  <!DOCTYPE html>
2  <html lang="pt-BR">
3  <head>
4  <meta charset="utf-8">
5  <title>Olá Senai</title>
6  </head>
7  <body>
8  <h1>olá Senai!</h1>
9  </body>
10 </html>
11
12
```

Dentro deste arquivo vamos codar o código html



```
receitas > templates > dj home.html
1  <!DOCTYPE html>
2  <html lang="pt-BR">
3  <head>
4  <meta charset="utf-8">
5  <title>Olá Senai</title>
6  </head>
7  <body>
8  <h1>olá Senai!</h1>
9  </body>
10 </html>
11
12
```



# Django – HTML

Se rodar a página vai dar erro, pois não informamos ao Django que existe este app.

## TemplateDoesNotExist at /

home.html

**Request Method:** GET

**Request URL:** http://127.0.0.1:8000/

**Django Version:** 5.2.7

**Exception Type:** TemplateDoesNotExist

**Exception Value:** home.html

**Exception Location:** f:\Senai\2025 - 2\FIC\Python como framework\ProjetoDjangoPY\venv\Lib\site-packages\django\template\load

**Raised during:** receitas.views.home

**Python Executable:** f:\Senai\2025 - 2\FIC\Python como framework\ProjetoDjangoPY\venv\Scripts\python.exe

**Python Version:** 3.13.1

**Python Path:** ['F:\\Senai\\2025 - 2\\FIC\\Python como framework\\ProjetoDjangoPY',  
'c:\\Users\\Professor\\.vscode\\extensions\\ms-python.debugpy-2025.14.1-win32-x64\\bundled\\libs\\debugpy\\',  
'C:\\Users\\Professor\\AppData\\Local\\Programs\\Python\\Python313\\python313.zip',  
'C:\\Users\\Professor\\AppData\\Local\\Programs\\Python\\Python313\\DLLs',  
'C:\\Users\\Professor\\AppData\\Local\\Programs\\Python\\Python313\\Lib',  
'C:\\Users\\Professor\\AppData\\Local\\Programs\\Python\\Python313',  
'f:\\Senai\\2025 - 2\\FIC\\Python como framework\\ProjetoDjangoPY\\venv',  
'f:\\Senai\\2025 - 2\\FIC\\Python como '  
'framework\\ProjetoDjangoPY\\venv\\Lib\\site-packages']

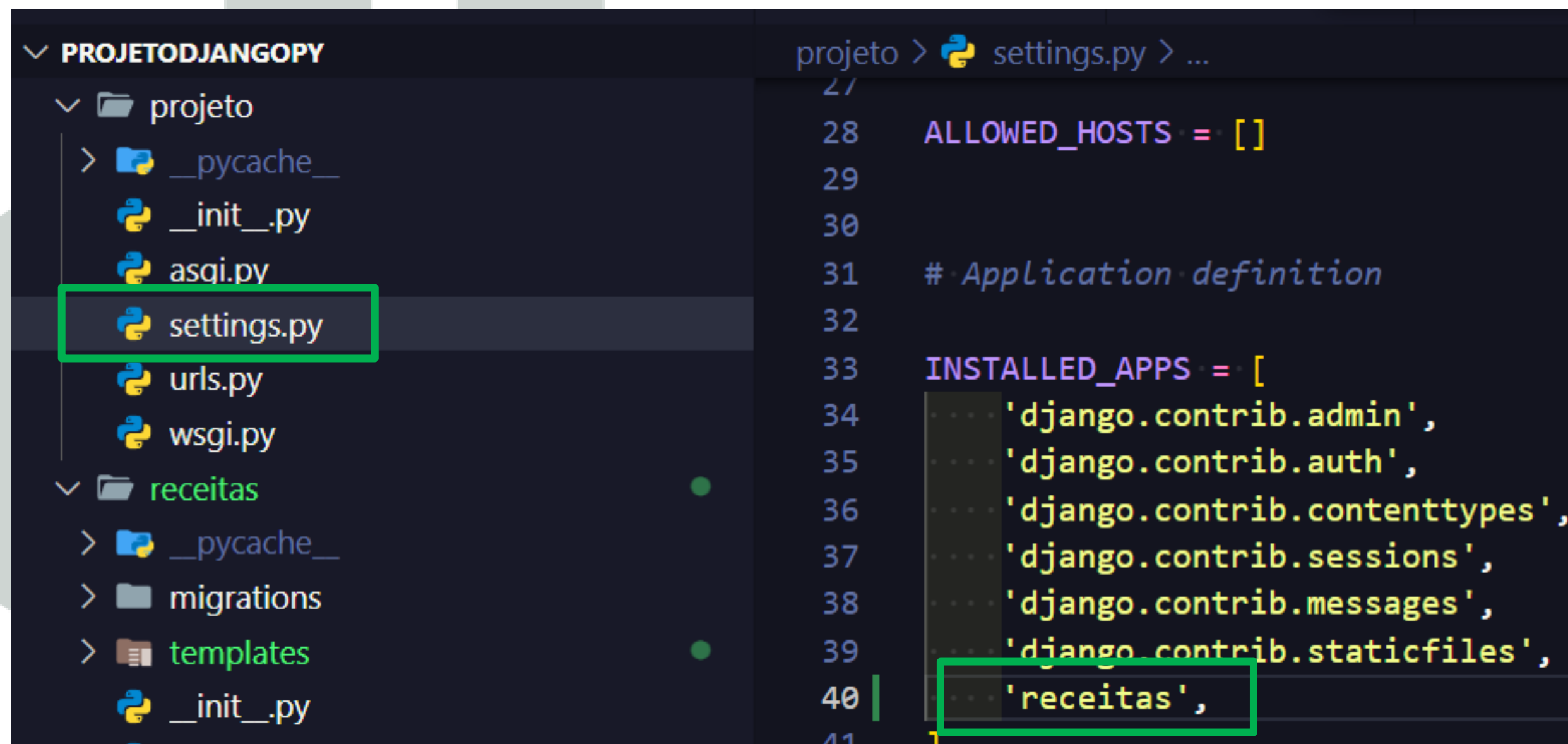
**Server time:** Wed, 22 Oct 2025 22:05:35 +0000

Template-loader postmortem



# Django – HTML

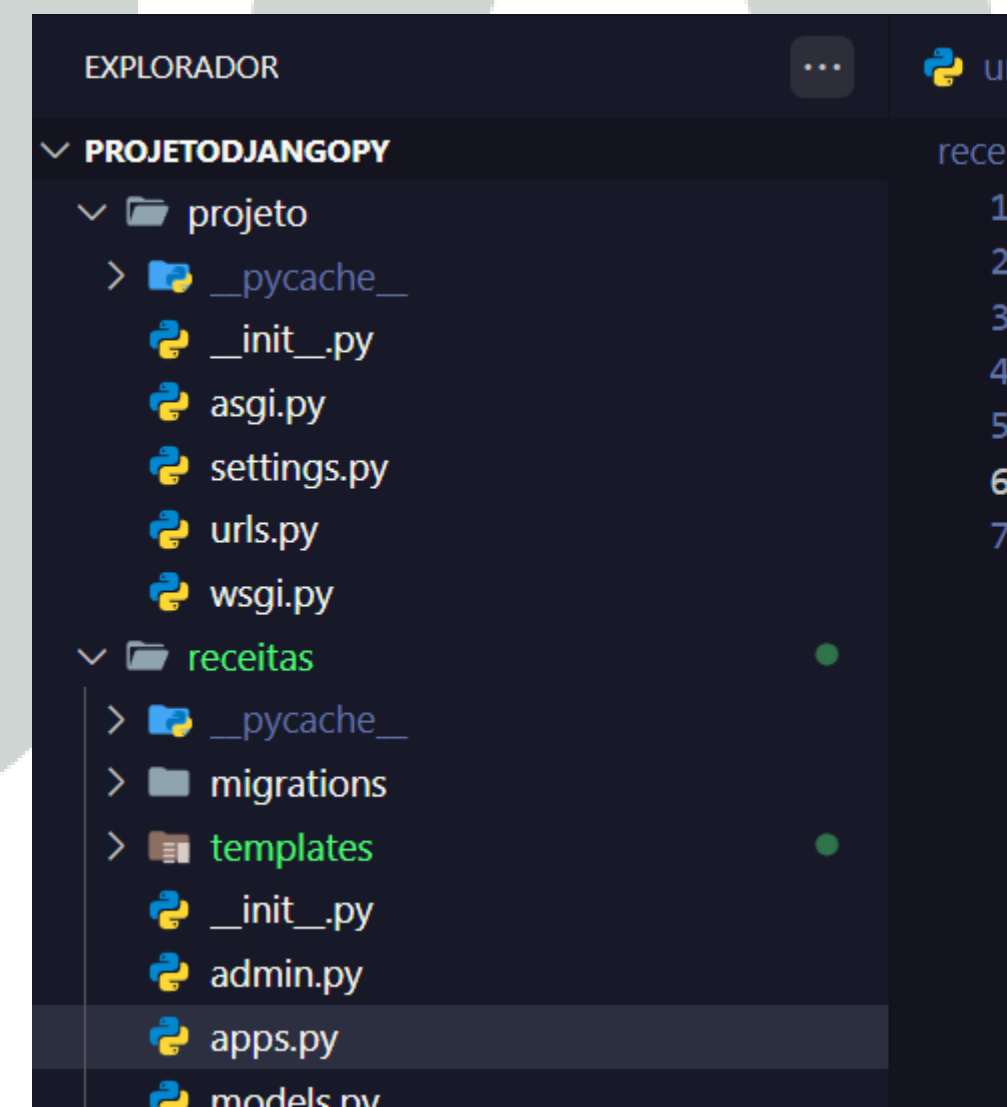
Para corrigir, ir no arquivo *settings.py* e inserir o nome do seu app



```

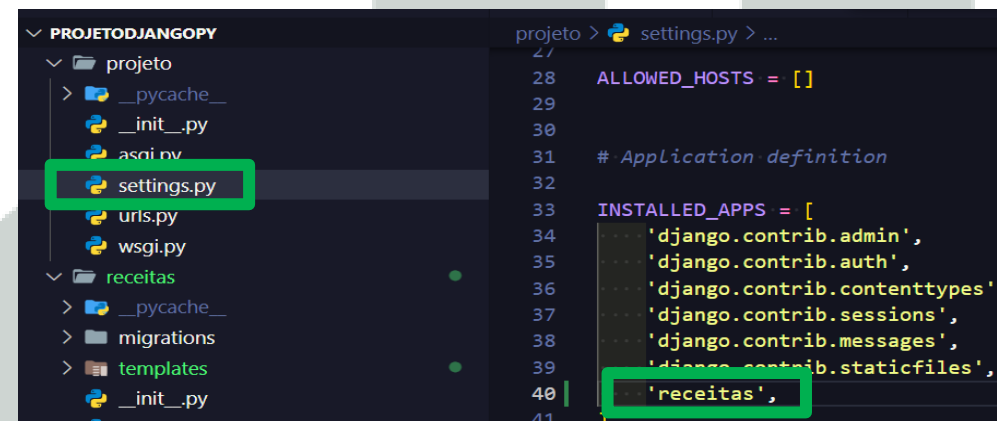
projeto > settings.py > ...
27
28 ALLOWED_HOSTS = []
29
30
31 # Application definition
32
33 INSTALLED_APPS = [
34     'django.contrib.admin',
35     'django.contrib.auth',
36     'django.contrib.contenttypes',
37     'django.contrib.sessions',
38     'django.contrib.messages',
39     'django.contrib.staticfiles',
40     'receitas',
41

```

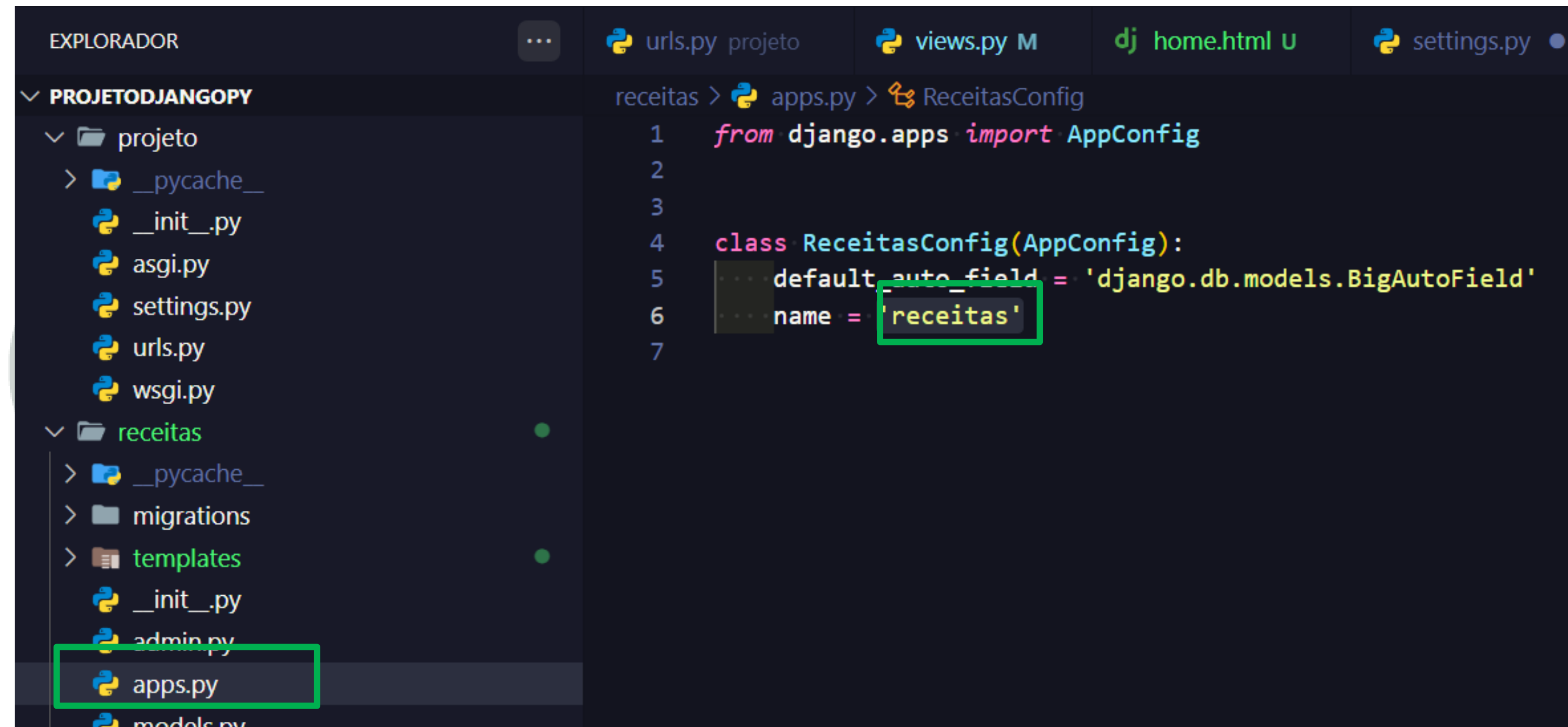


# Django – HTML

Para verificar o nome correto do seu app's, ir na pasta *app.py*



```
28 ALLOWED_HOSTS = []
29
30 # Application definition
31
32 INSTALLED_APPS = [
33     'django.contrib.admin',
34     'django.contrib.auth',
35     'django.contrib.contenttypes',
36     'django.contrib.sessions',
37     'django.contrib.messages',
38     'django.contrib.staticfiles',
39     'receitas',
40 ]
```



```
EXPLORADOR
PROJETODJANGOPY
projeto
  __pycache__
  __init__.py
  asgi.py
  settings.py
  urls.py
  wsgi.py
receitas
  __pycache__
  migrations
  templates
  __init__.py
  admin.py
  apps.py
  models.py

receitas > apps.py > ReceitasConfig
1 from django.apps import AppConfig
2
3
4 class ReceitasConfig(AppConfig):
5     default_auto_field = 'django.db.models.BigAutoField'
6     name = 'receitas'
7
```

Agora o código roda com HTML funcionando!



# Olá Senai!

Quebra de linha ☐

```
1 <!DOCTYPE html>
2     <html lang="pt-BR">
3         <head>
4             <meta charset="utf-8">
5             <title>Olá Senai</title>
6         </head>
7         <body>
8             <h1>Olá Senai!</h1>
9         </body>
10    </html>
```

# **Explorando um pouco a função *render***



# Django – Função render

Segurando o Ctrl e clicando encima do *render* podemos verificar o código desta função

```
1 from django.shortcuts import render
2 from django.http import HttpResponse
3
4 # Create your views here.
5 def home(request):
6     return render(request, 'home.html', context={
7         ... 'no'
8         ... })
9
10 def sobre(r
11     ... return
12
13 def contato
14     ... return
```

```
(function) def render(
    request: HttpRequest,
    template_name: str | Sequence[str],
    context: Mapping[str, Any] | None = ...,
    content_type: str | None = ...,
    status: int | None = ...,
    using: str | None = ...
) -> HttpResponse
```

Return an HttpResponse whose content is filled with the result of calling `django.template.loader.render_to_string()` with the passed arguments.

[Veja exemplos do mundo real do GitHub](#)

```
def render(
    request, template_name, context=None, content_type=None, status=None, using=None
):
    """
    Return an HttpResponse whose content is filled with the result of calling
    django.template.loader.render_to_string() with the passed arguments.
    """
    content = loader.render_to_string(template_name, context, request, using=using)
    return HttpResponse(content, content_type, status)
```



# Django – Função render

Nesta função, podemos verificar que temos 6 argumentos que podemos passar para dentro da função

```
def render(  
    request, template_name, context=None, content_type=None, status=None, using=None  
):  
    """  
    Return an HttpResponse whose content is filled with the result of calling  
    django.template.loader.render_to_string() with the passed arguments.  
    """  
    content = loader.render_to_string(template_name, context, request, using=using)  
    return HttpResponse(content, content_type, status)
```

# Django – Função render

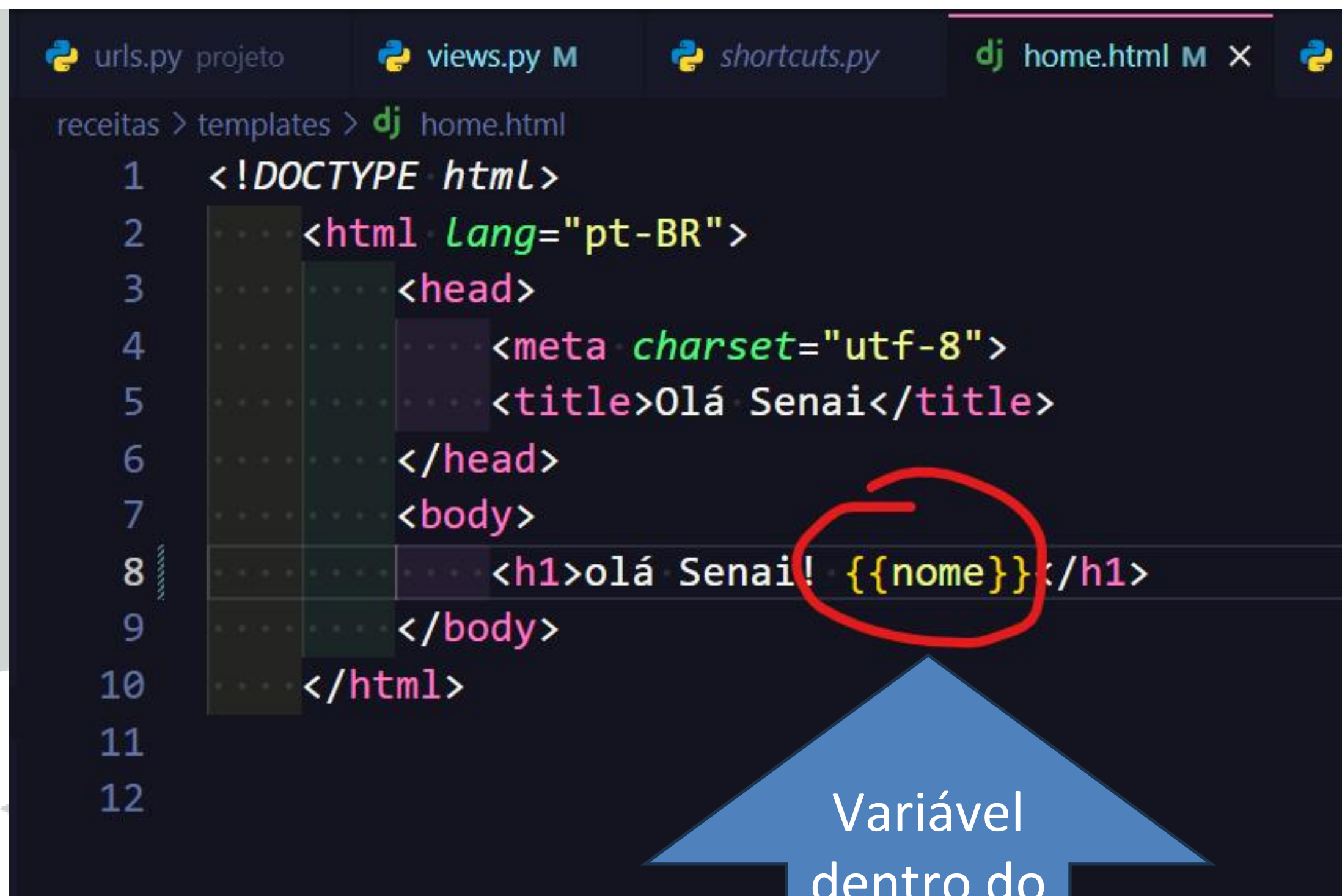
Se na função inserir um dicionário, podemos colocar variáveis dentro do código HTML

```
# Create your views here.  
def home(request):  
    return render(request, 'home.html', context={  
        'nome': 'Receitas Django'  
    })
```

Coleção  
do tipo  
dicionário  
python

# Django – Função render

E ao ir para o código html, podemos inserir esta chave dentro do código



```
urls.py projeto  views.py M  shortcuts.py  dj home.html M x
receitas > templates > dj home.html
1  <!DOCTYPE html>
2  <html lang="pt-BR">
3  <head>
4  <meta charset="utf-8">
5  <title>Olá Senai</title>
6  </head>
7  <body>
8  <h1>olá Senai! {{nome}}</h1>
9  </body>
10 </html>
11
12
```

Variável  
dentro do  
código  
html

Com isso abrimos várias  
possibilidades dentro do  
Django.



A nighttime photograph of a city street, likely in São Paulo, featuring light trails from moving vehicles and illuminated buildings. A red rectangular logo with the word "SENAI" in white, bold, sans-serif capital letters is overlaid on the left side of the image. The logo has horizontal white lines on either side of the text.

**SENAI**

DEPARTAMENTO REGIONAL  
DE SÃO PAULO

[www.sp.senai.br](http://www.sp.senai.br)